

Proyecto 1

Orlando Almazán Gallardo, 198046

Adrián Janus González Adamuz, 202039

Valentina Márquez Gil, 194788

Adrian Dominguez Olvera, 197122

Diego Hinojosa Téllez, 189958

10 de octubre de 2025

Profesora: Dr.Airam Blancas

Pregunta 1

Sea una cadena de Markov $(X_n, n \in \mathbb{N}_0)$ con espacio de estados

$$E = \{0, 1, 2, \dots, N\}, \quad \text{donde } N \leq \infty.$$

Se denomina *cadena de nacimiento y muerte* si su matriz de transición está dada por

$$P_{ij} = \begin{cases} q_i, & \text{si } j = i - 1, \\ r_i, & \text{si } j = i, \\ p_i, & \text{si } j = i + 1, \\ 0, & \text{en otro caso,} \end{cases}$$

con $p_i, q_i, r_i \in (0, 1)$ para cada $i \in E$, y $q_0 = p_N = 0$ cuando $N < \infty$.

(a) Irreducibilidad y Análisis del Primer Paso

Supongamos que $N < \infty$, $p_0 > 0$, $q_N > 0$, $p_j > 0$ y $q_j > 0$ para todo $j \in \{1, \dots, N-1\}$. Demuestra que la cadena es irreducible y utiliza el análisis del primer paso para encontrar $\mathbb{P}_k(H_a < H_b)$ para todo $a, b, k \in E$ tal que $a < k < b$.

Demostración de Irreducibilidad

Una cadena de Markov es irreducible si todos sus estados se comunican entre sí. Debemos demostrar que para cualquier par de estados $i, j \in E = \{0, 1, \dots, N\}$, se cumple que $i \leftrightarrow j$.

Las condiciones dadas son que las probabilidades de nacimiento $p_k = P_{k,k+1} > 0$ para $k \in \{0, \dots, N-1\}$ y las probabilidades de muerte $q_k = P_{k,k-1} > 0$ para $k \in \{1, \dots, N\}$.

1. **Trayectoria de i a j con $i < j$:** Podemos tomar la trayectoria de nacimientos consecutivos: $i \rightarrow i+1 \rightarrow \dots \rightarrow j$. La probabilidad de esta trayectoria de $n = j - i$ pasos es:

$$P(X_n = j | X_0 = i) \geq p_i \cdot p_{i+1} \cdot \dots \cdot p_{j-1}$$

Dado que todos los $p_k > 0$ en este rango, el producto es mayor que cero. Por lo tanto, el estado j se accede desde i ($i \rightarrow j$).

2. **Trayectoria de i a j con $i > j$:** Análogamente, podemos tomar la trayectoria de muertes consecutivas: $i \rightarrow i-1 \rightarrow \dots \rightarrow j$. La probabilidad de esta trayectoria de

$n = i - j$ pasos es:

$$P(X_n = j | X_0 = i) \geq q_i \cdot q_{i-1} \cdot \dots \cdot q_{j+1}$$

Dado que todos los $q_k > 0$ en este rango, el producto es mayor que cero. Por lo tanto, $i \rightarrow j$.

Como es posible ir de cualquier estado i a cualquier otro estado j y viceversa, todos los estados se comunican. Concluimos que la cadena es **irreducible**.

Cálculo de $\mathbb{P}_k(H_a < H_b)$

Sea $h_k = \mathbb{P}_k(H_a < H_b)$ la probabilidad de que la cadena, partiendo de $X_0 = k$, alcance el estado a antes que el estado b , con $a < k < b$. Notamos que $h_a = 1$ y $h_b = 0$.

Por análisis del primer paso, para $a < k < b$:

$$\begin{aligned} h_k &= p_k h_{k+1} + q_k h_{k-1} + r_k h_k \\ h_k(1 - r_k) &= p_k h_{k+1} + q_k h_{k-1} \\ h_k(p_k + q_k) &= p_k h_{k+1} + q_k h_{k-1} \\ p_k(h_{k+1} - h_k) &= q_k(h_k - h_{k-1}) \end{aligned}$$

Sea la diferencia $d_k = h_k - h_{k-1}$. La recurrencia es $p_k d_{k+1} = q_k d_k$, o $d_{k+1} = \frac{q_k}{p_k} d_k$. Resolviendo esta recurrencia, obtenemos $d_k = d_{a+1} \prod_{j=a+1}^{k-1} \frac{q_j}{p_j}$.

Mediante una suma telescópica, $h_k - h_a = \sum_{i=a+1}^k d_i$. Como $h_a = 1$:

$$h_k = 1 + d_{a+1} \sum_{i=a+1}^k \left(\prod_{j=a+1}^{i-1} \frac{q_j}{p_j} \right)$$

Usamos $h_b = 0$ para encontrar d_{a+1} :

$$0 = 1 + d_{a+1} \sum_{i=a+1}^b \left(\prod_{j=a+1}^{i-1} \frac{q_j}{p_j} \right) \implies d_{a+1} = - \left(\sum_{i=a+1}^b \prod_{j=a+1}^{i-1} \frac{q_j}{p_j} \right)^{-1}$$

Sustituyendo d_{a+1} de vuelta, la solución final es:

$$\mathbb{P}_k(H_a < H_b) = 1 - \frac{\sum_{i=a+1}^k \prod_{j=a+1}^{i-1} \frac{q_j}{p_j}}{\sum_{i=a+1}^b \prod_{j=a+1}^{i-1} \frac{q_j}{p_j}} = \frac{\sum_{i=k+1}^b \prod_{j=a+1}^{i-1} \frac{q_j}{p_j}}{\sum_{i=a+1}^b \prod_{j=a+1}^{i-1} \frac{q_j}{p_j}}$$

donde un producto vacío (cuando $i = a + 1$) se define como 1.

(b) Caminata Aleatoria Simple con Barreras Absorbentes

Utiliza el resultado obtenido en el inciso anterior para obtener $\mathbb{P}_k(H_N < H_0)$ en una caminata aleatoria simple con barreras absorbentes.

Queremos encontrar $\mathbb{P}_k(H_N < H_0)$, la probabilidad de ser absorbido en la barrera superior N antes que en la inferior 0. Este es un caso especial del problema de absorción. Usamos la fórmula análoga a la del inciso (a) para la probabilidad de alcanzar b antes que a :

$$\mathbb{P}_k(H_b < H_a) = \frac{\sum_{i=a+1}^k \prod_{j=a+1}^{i-1} \frac{q_j}{p_j}}{\sum_{i=a+1}^b \prod_{j=a+1}^{i-1} \frac{q_j}{p_j}}$$

Para nuestro caso, los límites son $a = 0$ y $b = N$. Para una caminata aleatoria simple, las probabilidades de transición son constantes: $p_j = p$ y $q_j = q$. El cociente es $\gamma = q/p$.

La fórmula se simplifica a:

$$\mathbb{P}_k(H_N < H_0) = \frac{\sum_{i=1}^k \gamma^{i-1}}{\sum_{i=1}^N \gamma^{i-1}} = \frac{\sum_{j=0}^{k-1} \gamma^j}{\sum_{j=0}^{N-1} \gamma^j}$$

Resolvemos esta expresión para los dos casos posibles.

Caso 1: Caminata simétrica ($p = q = 1/2$)

En este caso, $\gamma = 1$. La suma de una serie de unos, $\sum_{j=0}^{m-1} 1 = m$. Por lo tanto:

$$\mathbb{P}_k(H_N < H_0) = \frac{k}{N}$$

Caso 2: Caminata asimétrica ($p \neq q$)

En este caso, $\gamma \neq 1$. Usamos la fórmula para la suma de una serie geométrica, $\sum_{j=0}^{m-1} \gamma^j = \frac{1-\gamma^m}{1-\gamma}$.

$$\mathbb{P}_k(H_N < H_0) = \frac{(1-\gamma^k)/(1-\gamma)}{(1-\gamma^N)/(1-\gamma)} = \frac{1-\gamma^k}{1-\gamma^N}$$

Sustituyendo de vuelta $\gamma = q/p$:

$$\mathbb{P}_k(H_N < H_0) = \frac{1-(q/p)^k}{1-(q/p)^N}$$

(c) Simulación de una Cadena de Nacimiento y Muerte

Simula una cadena de nacimiento y muerte.

La simulación de una cadena de Markov es una implementación computacional directa de su definición fundamental: la **propiedad de Markov**. Esta propiedad establece que el futuro del proceso, dado el presente, es independiente del pasado.

El algoritmo de simulación funciona de la siguiente manera:

1. Se inicia en un estado $X_0 = k$.
2. Para cada paso n , se observa el estado actual $X_n = i$.
3. Se utiliza la fila i de la matriz de transición, $(P_{i,j})_{j \in E}$, para determinar el siguiente estado X_{n+1} . Esto se hace generando un número aleatorio uniforme y viendo en qué intervalo de probabilidad cae, correspondiendo a una transición a $i - 1$ (muerte), i (estasis), o $i + 1$ (nacimiento).
4. El proceso se repite, y en cada paso, la elección del siguiente estado depende únicamente del estado actual, respetando la propiedad de Markov. (Codigo en el anexo)

Despues de hacer una simulación nos quedo la siguiente cadena

Grafo del Espacio de Estados de la Cadena de Nacimiento y Muerte

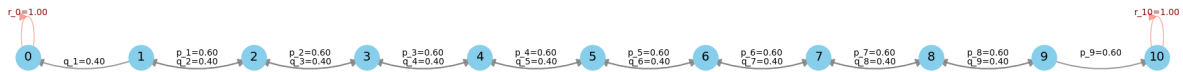


Figura 1: Grafo del Espacio de Estados de la Cadena de Nacimiento y Muerte.

Pregunta 2

Las caminatas aleatorias juegan un papel central en la teoría de probabilidad. Define tres ejemplos distintos de caminatas aleatorias y explica brevemente su relevancia

Ejemplo 1: Caminata aleatoria en grupo - problema de barajar cartas

¿Qué es un Grupo?

En álgebra abstracta, un grupo es una de las estructuras más fundamentales. De manera sencilla, un grupo es un conjunto de elementos junto con una operación que permite combinar dos elementos para obtener un tercero. Para ser un grupo, debe cumplir cuatro reglas básicas:

- **Cierre (o Clausura):** Si tomas dos elementos cualesquiera del conjunto y los operas, el resultado siempre es otro elemento que también pertenece al conjunto.
- **Elemento Identidad (o Neutro):** Existe un elemento especial en el conjunto (llamado e) que, al operarlo con cualquier otro elemento, no lo modifica. Por ejemplo, para la suma, el 0 es la identidad ($a + 0 = a$).
- **Elemento Inverso:** Para cada elemento del conjunto, existe un elemento inverso que, al operarlos juntos, dan como resultado el elemento identidad ($a \cdot a^{-1} = e$).
- **Asociatividad:** No importa cómo agrupes los elementos al operarlos, el resultado final es el mismo. Es decir, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.

El ejemplo más simple es el conjunto de los números enteros ($\mathbb{Z}$) con la operación de suma (+).

Propiedades Principales de la Caminata

Una **caminata aleatoria en grupos** es un proceso estocástico en el que los estados posibles son los elementos de un grupo finito G . La idea es que partimos de un elemento inicial g_0 y, en cada paso, multiplicamos el estado actual por un elemento aleatorio del grupo elegido según una distribución de probabilidad μ . Esto genera una sucesión de estados $X_0, X_1, X_2, \dots$. Sus propiedades principales son:

- **Espacio de estados:** los elementos del grupo G .
- **Evolución:** $X_{t+1} = Y_{t+1} \cdot X_t$.
- **Matriz de transición:** $P(g, h) = \mu(hg^{-1})$.

- **Distribución estacionaria:** la uniforme sobre G .

Relevancia del Modelo

Este modelo de caminata aleatoria es relevante porque traduce un problema físico (barajar cartas) a un problema matemático preciso y elegante, permitiendo analizarlo con herramientas poderosas.

- **Los estados son los ordenamientos:** Cada elemento del grupo G representa un posible ordenamiento (o permutación) del mazo de cartas. El grupo de todas las permutaciones de n cartas se conoce como el grupo simétrico S_n .
- **La operación es "barajar":** La multiplicación por un elemento aleatorio del grupo (Y_{t+1}) equivale a aplicar un paso del barajeo. Por ejemplo, si el barajeo consiste en intercambiar dos cartas al azar, entonces Y_{t+1} sería la transposición correspondiente a ese intercambio.
- **El objetivo es la aleatoriedad total:** La distribución estacionaria uniforme significa que, después de muchos pasos, todos los posibles ordenamientos del mazo son igualmente probables. Esto es, por definición, un mazo perfectamente barajado.
- **El tiempo de mezcla responde la pregunta clave:** El "tiempo de mezcla" nos dice cuántos pasos (barajeos) son necesarios para que la distribución de los posibles ordenamientos del mazo sea lo suficientemente cercana a la distribución uniforme. En términos prácticos, responde a la pregunta: *¿cuántas veces debo barajar para que el mazo esté bien revuelto?*

Características del Modelo

Entonces, considerando esto, analicemos este modelo. Sus características son:

- **Espacio de estados:** El espacio está dado por $\omega = G$, es decir, cada estado corresponde a un elemento del grupo. Una caminata en G puede entenderse como una sucesión (X_t) donde

$$X_0 = g_0, \quad X_{t+1} = Y_{t+1} \cdot X_t,$$

siendo Y_t variables aleatorias independientes con distribución μ .

- **Matriz de transición:** En cada paso, el estado actual se multiplica (por la izquierda) con un elemento aleatorio elegido de acuerdo con μ . Formalmente, la matriz de transición está dada por

$$P(g, h) = \mu(hg^{-1}), \quad \forall g, h \in G.$$

Esto significa que la probabilidad de pasar de g a h depende únicamente del incremento $g^{-1}h$.

- **Distribución estacionaria:** La distribución estacionaria sobre G resulta ser la distribución uniforme. Esto se debe a la simetría del grupo: al multiplicar uniformemente por cualquier elemento, la uniformidad se preserva. Es decir,

$$U = UP$$

donde U es la distribución uniforme en G .

Ejemplo 2: Caminata aleatoria en mercados financieros

La teoría de la caminata aleatoria en el ámbito financiero sostiene que los precios de las acciones son independientes entre sí y comparten una misma distribución; esto implica que la información del comportamiento pasado de los precios **no** sirve para predecir su evolución futura. Este modelo está diseñado precisamente para capturar esos movimientos impredecibles en los mercados financieros. Para ello, se generaliza la idea de caminata aleatoria hacia un proceso estocástico discreto que modela la evolución temporal de los precios, interpretando cada cambio como un incremento aleatorio.

Sea P_t el precio de un activo en el instante t , modelado como una sucesión $\{P_0, P_1, P_2, \dots\}$, donde cada variación (absoluta o porcentual) es una variable aleatoria con media cero (es decir, sin tendencia direccional).

Espacio de estados

El espacio de precios puede considerarse como $\Omega = \mathbb{R}^+$ (valores positivos continuos), o en una versión discreta como $\Omega = \{P_{\min}, \dots, P_{\max}\}$ para simular cotizaciones dentro de un rango acotado. Una caminata en este espacio se define como un proceso (X_t) con $X_0 = p_0$ y

$$X_{t+1} = X_t + \epsilon_t,$$

donde ϵ_t son variables aleatorias independientes con distribución μ , cuyo soporte podría ser, por ejemplo, $\{-d, 0, +d\}$ (es decir, subida, bajada o estabilidad con ciertas probabilidades).

Matriz de transición

En cada paso, el precio transita según un incremento aleatorio tomado de μ . Formalmente, la probabilidad de transitar del estado i al estado j viene dada por

$$P(i, j) = \mu(j - i), \quad \forall i, j \in \Omega,$$

siempre que $j - i$ pertenezca al soporte de μ . En otras palabras, la probabilidad de moverse de i a j depende únicamente del “incremento” $j - i$.

Distribución estacionaria

Cuando el espacio es ilimitado (por ejemplo $\Omega = \mathbb{Z}$), la caminata aleatoria simple no posee distribución estacionaria, pues es un proceso transiente. Sin embargo, si restringimos los precios a un intervalo finito $[L, U]$ con condiciones de reflexión o absorción en los límites, la distribución estacionaria tiende a aproximarse a una distribución uniforme sobre los precios alcanzables, gracias a la simetría y la estructura homogénea del espacio.

Ejemplo de aplicación

Consideremos el precio de una acción P_t a lo largo del tiempo $t = 0, 1, 2, \dots$. Un modelo simple de caminata aleatoria asume que los precios evolucionan según

$$P_{t+1} = P_t + \varepsilon_t,$$

donde ε_t representa un cambio aleatorio independiente con $\mathbb{E}[\varepsilon_t] = 0$ y $\text{Var}(\varepsilon_t) = \sigma^2$. Esta formulación implica que el mejor pronóstico del precio futuro, dado el pasado, es simplemente el precio actual:

$$\mathbb{E}[P_{t+1} \mid P_t, P_{t-1}, \dots] = P_t.$$

En otras palabras, los precios siguen un proceso *martingala*, lo que significa que las variaciones pasadas no contienen información útil para predecir las futuras.

En la práctica, este modelo se aplica frecuentemente al análisis de rendimientos logarítmicos, definidos como

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right).$$

Bajo la hipótesis de caminata aleatoria, los rendimientos $\{r_t\}$ constituyen una secuencia de variables aleatorias independientes e idénticamente distribuidas (i.i.d.) con media cero:

$$r_t \sim \text{i.i.d. } (0, \sigma^2).$$

Por ejemplo, si un activo financiero tiene una volatilidad diaria aproximada de $\sigma = 0,015$, la desviación estándar acumulada de los precios después de n días será proporcional a $\sqrt{n}\sigma$, reflejando la naturaleza difusiva del proceso.

El modelo de caminata aleatoria permite capturar la idea de que los precios en los mercados financieros se comportan de manera impredecible, reflejando la incorporación instantánea de toda la información disponible. Aunque es un modelo simplificado, proporciona una base teórica para el estudio de la eficiencia de los mercados y del riesgo financiero. Además, puede extenderse fácilmente a versiones más realistas, como los modelos de difusión continua

$$dP_t = \mu P_t dt + \sigma P_t dW_t,$$

donde W_t es un movimiento browniano estándar, μ representa la tasa de crecimiento esperada y σ la volatilidad instantánea del activo.

Ejemplo 3: Caminata aleatoria en grafos dirigidos

Sea $G = (V, E)$ un grafo finito dirigido tal que $|G| = n$. Un camino es una sucesión de vértices adyacentes $v_1 v_2 \dots v_m \dots$, donde $l_i = (v_i, v_{i+1})$ $l_i \in E \forall i = 1, 2, \dots$.

Una caminata aleatoria en G es un camino que empieza en un vértice $X_0 = v_i$ y la probabilidad de pasar a un vértice al otro sólo depende del vértice en el que está la caminata en el presente. Consideramos un modelo donde la probabilidad de pasar a todos los vecinos es igual, esto es:

$$P(X_{m+1} = v_j \mid X_m = v_i) = \begin{cases} \frac{1}{d(v_i)}, & \text{si } (v_i, v_j) \in E, \\ 0, & \text{en otro caso.} \end{cases}$$

donde $d(v_i)$ es el grado de salida de v_i . Ahora podemos definir a los vértices como estados y, como G es finito, estas definiciones inducen una matriz de transición P .

Podemos concluir que (X_k) es una cadena de Markov.

PageRank

PageRank es un algoritmo desarrollado por Larry Page y Serguéi Brin, fundadores de Google, para encontrar una jerarquía en las páginas de internet. La idea innovadora que tuvieron fue modelar la red de páginas de internet como un grafo dirigido de esta forma:

- El conjunto de páginas de internet es un grafo $G = (V, E)$.

- Cada página es un nodo.
- La arista $(v_i, v_j) \in E$ si la página v_i tiene un enlace que dirige a v_j .
- Un *random surfer* empieza una caminata aleatoria en G .

Después de muchas vistas, el algoritmo consigue la distribución de probabilidad de estar en cualquier página. Eventualmente, el PageRank de una página corresponde a la probabilidad estacionaria de esta cadena de Markov:

$$\pi = \pi P$$

En el caso de las caminatas aleatorias, la distribución de probabilidad sobre G para cada nodo v_i en cualquier tiempo dado representa la probabilidad de que la caminata aleatoria se encuentre en el nodo v_i .

En la forma más sencilla, distribución inicial de todos los vértices era igual. Si empezamos con una distribución π , $\pi_t = \pi P^t$, cuando hay suficientes pasos o iteraciones del proceso, llegamos a una distribución estacionaria que cumple $\pi_e = \pi_e P^t$ que ya no cambia y determina la probabilidad de que esté el *random surfer* esté en una página (o vértice de G) de interés.

Este algoritmo consiguió encontrar las páginas más relevantes y catapultó a Google como el buscador más acertado y una de las compañías más innovadoras del mundo, aunque no es lo único que puede hacer. Las redes digitales aprovechan estas ideas para encontrar personas con intereses parecidos. En la neurociencia se estudian las relaciones dirigidas que existen en el cerebro para entender su arquitectura. PageRank demostró cómo un concepto tan teórico como una caminata aleatoria puede tener un impacto profundo en el mundo real. La combinación de probabilidad, teoría de grafos y procesos estocásticos permitió construir una herramienta capaz de ordenar la información, y sus principios siguen inspirando soluciones en ciencia de datos, biología, redes sociales y otros sistemas .

Pregunta 3

Construye un modelo basado en una cadena de Markov para una situación de interés. Especifica claramente el espacio de estados y la matriz de transición.

Se construyó un modelo utilizando *cadena de Markov* de la melodía de la primera balada escrita por el compositor polaco Frédéric Chopin terminada en 1835 (Balada no. 1 en Sol menor, op. 23). La cadena de Markov tiene los siguientes elementos:

1. El **espacio de estados** son 67 de las 88 teclas del piano, que van desde el Si 2 (B2) hasta el Fa 7 (F7), pasando por todas las teclas intermedias, contando obviamente los sostenidos/bemoles. Este espacio de estados no siempre es igual. Dependiendo de la pieza se tendrán usualmente menos de 67 teclas. Si en una pieza no se toca una tecla simplemente dicha tecla no pertenecerá al espacio de estados.
2. La **matriz de transiciones** consiste de las probabilidades de transicionar de un estado a otro (de una tecla a otra). Ésta probabilidad se obtiene con la proporción del número de veces que una tecla va hacia otra.

Para ejemplificar esto último, considere que en una pieza musical aparece cuatro veces el Do central (C4) y además considere que las notas que siguen de ese C4, para cada uno de los cuatro casos, son: Mi 4 (E4), Sol 5 (G5), Mi 4 (E4) y Do 5 (C5). Entonces, en este caso se tendría que la probabilidad de transicionar desde C4 a E4 es de $1/2$, la probabilidad de ir de C4 a G5 es $1/4$, de C4 a C5 es $1/4$ y la probabilidad de ir de C4 a cualquier otra nota es 0. Así, el renglón correspondiente a C4 se vería de la siguiente manera:

	otras	E4	G5	C5
C4	0	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$

Antes de mostrar los resultados es importante hacer algunas aclaraciones acerca de cómo se obtuvieron. Para comenzar, hay que mencionar que únicamente se está considerando la melodía de la pieza, la cual en el piano usualmente se toca con la mano derecha. Por ello, en este trabajo solo se utilizó la parte de la partitura correspondiente a la mano derecha (que en la mayoría de los casos se encuentra en clave de Sol). Otra cuestión relevante es que cuando en la mano derecha se presenta un acorde (un conjunto de tres notas, o más, tocadas al mismo tiempo) para el estudio se considera como si fuese un arpeggio ascendente (por ejemplo, si se toca el acorde de Do mayor C4-E4-G4 entonces se considera que C4 es la primera nota, que luego se toca E4 y luego G4). Además, hay que mencionar que, como se dijo, el espacio de estados solo toma en cuenta el rango de notas que va desde B2 a F7 (que son 67 teclas), esto es para eliminar las teclas de los extremos del piano (sobre todo el extremo izquierdo) que

usualmente no se utilizan en la melodía.

En cuanto a cómo se obtuvieron las transiciones, ya existen algunos códigos que contabilizan y clasifican las notas que hay en una pieza musical. Cada pieza puede ser codificada en formato *.mxl* que codifica las partituras a un lenguaje que es fácil de analizar para la computadora. Una vez obtenido un vector con todas las notas que contiene la pieza es relativamente sencillo realizar un código que contabilice las transiciones y con ello obtenga las proporciones que se utilizarán como las probabilidades de la matriz de transición.

Para esta pieza de Chopin, se tienen 53 estados, que corresponden a las notas que se tocan en la mano derecha. Se tienen, además, las siguientes frecuencias:

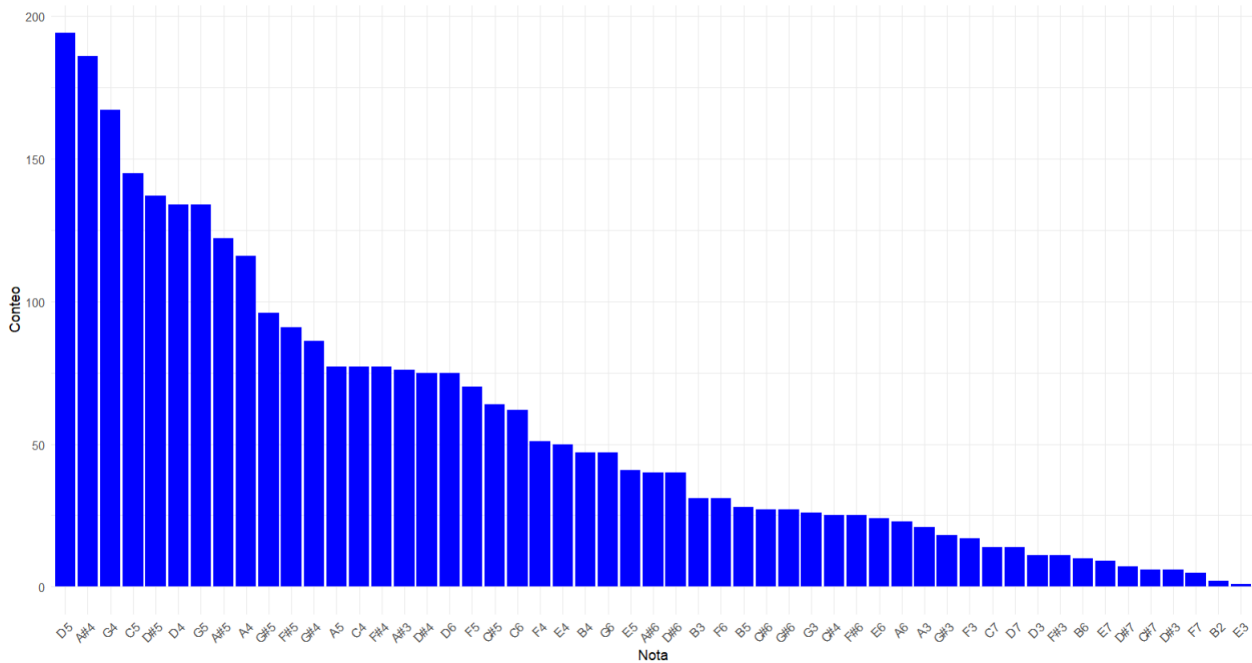


Figura 2: Gráfica de la frecuencia de notas

Puede verse que la nota que más se toca es el D5 (Re 5), que se toca 194 veces, seguida del A# (La sostenido 4, que en este caso en realidad es un Si bemol), que se toca 186 veces, y así sucesivamente, hasta llegar a que la nota menos tocada (sin contar obviamente a las que no se tocaron) es E3 (Mi 3), que solo se toca una vez.

Una vez se obtuvieron las proporciones de transición se construye la matriz de probabilidades de transición, la cual se ve de la siguiente manera:

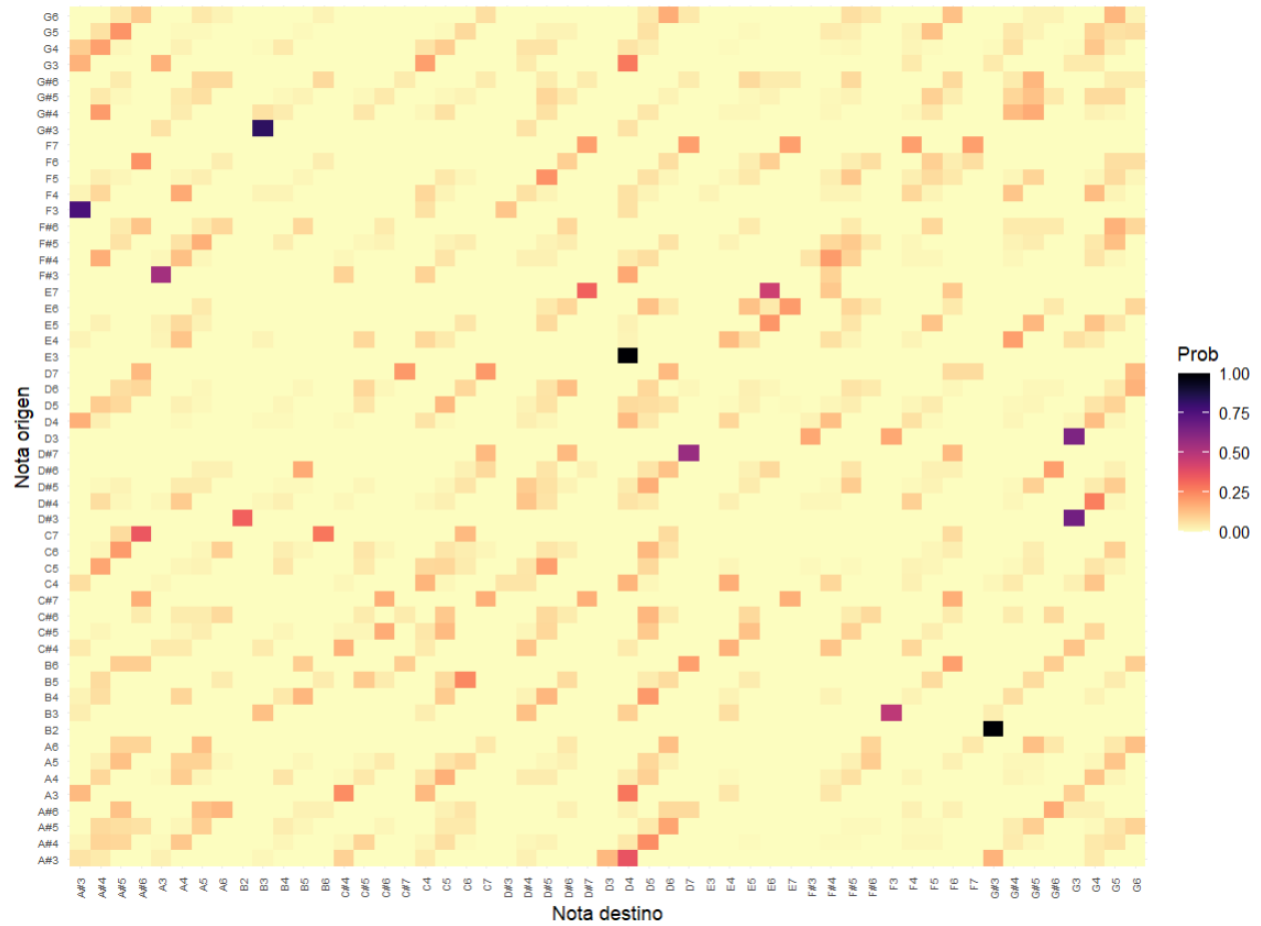


Figura 3: Grafo 1

Al ser una matriz relativamente grande (53x53) visualizar los números de la matriz es complicado. Por ello se muestra con colores, en donde en negro indica una probabilidad 1 y amarillo claro probabilidad 0. Note que hay muchos ceros en esta matriz. Además, se obtuvieron dos tipos de grafo en donde se muestran las transiciones:

Grafo de Transiciones (Layout Kamada-Kawai)

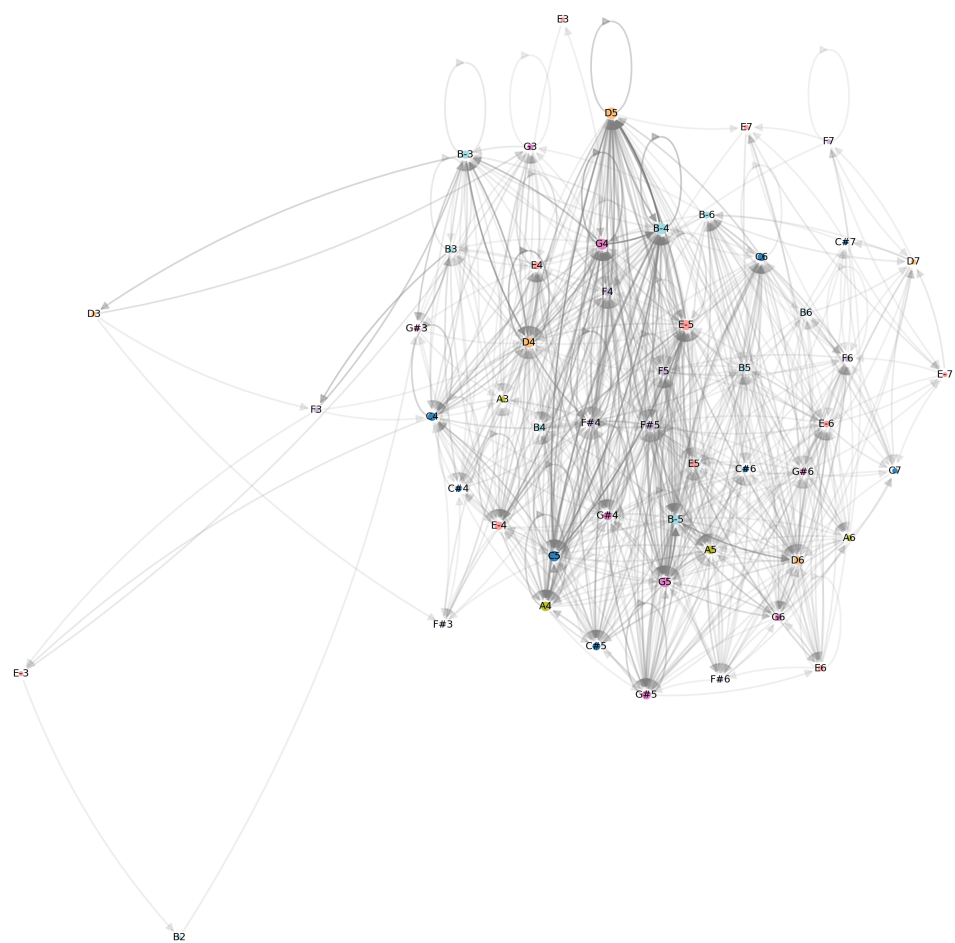


Figura 4: Grafo 1

Grafo de Transiciones (Layout Circular Musical)

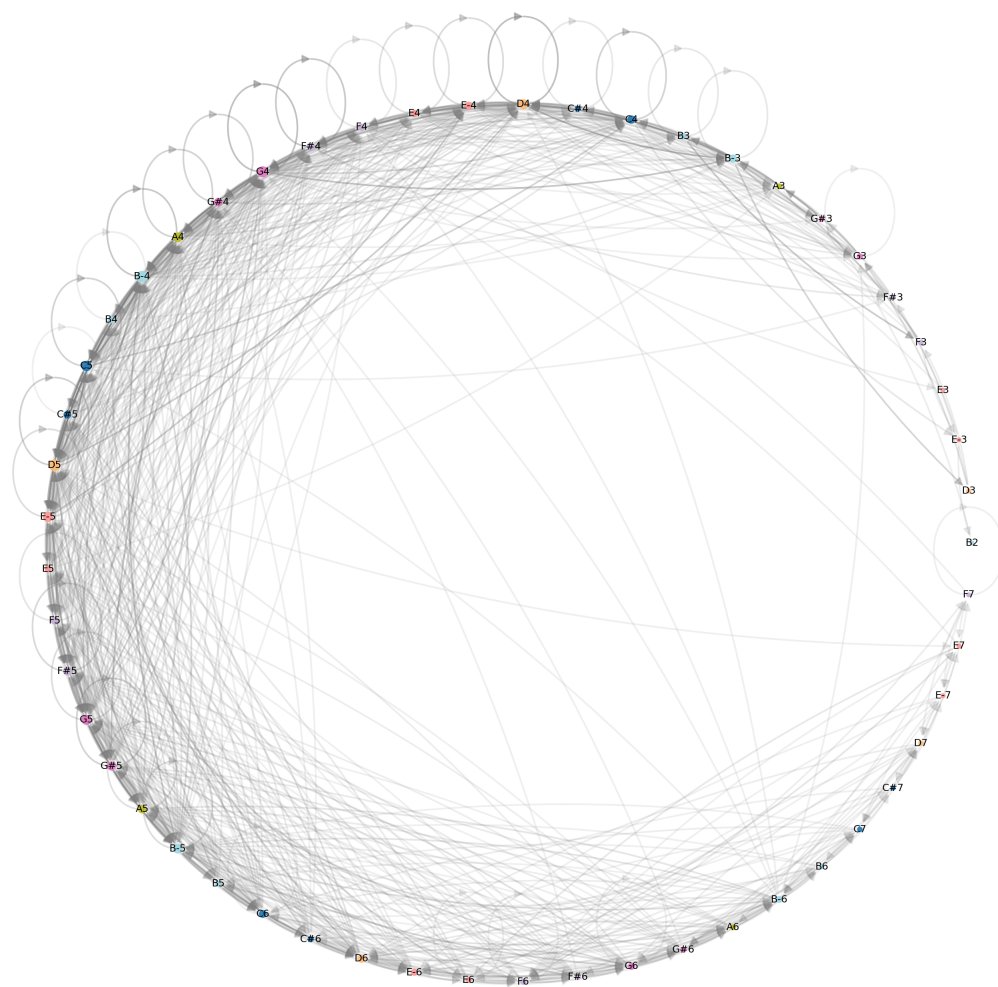


Figura 5: Grafo 2

Estudiando las clases de comunicación, se obtuvo que solo existe una clase cerrada, es decir, que todos los estados se comunican entre sí. Por ello, existe una única distribución estacionaria, la cual es:

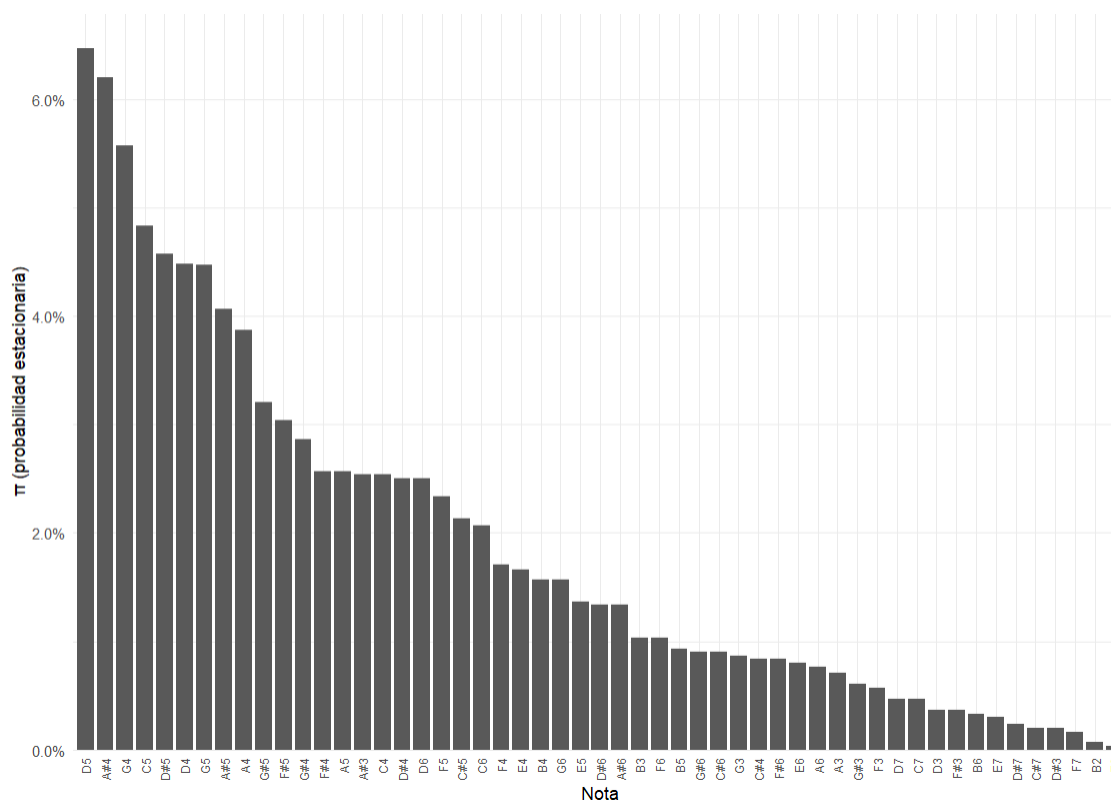


Figura 6: Distribución estacionaria

En lectura musical, esta distribución estacionaria revela el "peso estructural" que tiene cada nota y, con ello, la sensación de centro y reposo de la melodía. Esto es: a qué notas suele regresar la mayoría del tiempo la melodía (las notas más "pegajosas" de la melodía). Al estar la pieza compuesta en Sol menor se esperaría que estas notas sean aquellas que pertenecen a tal tonalidad (G, A, Eb, C, D, Eb, F, G). Note que en esta distribución estacionaria las notas con mayor masa de probabilidad son aquellas que conforman el acorde de Sol menor (G-Bb-D) (aquí Bb se codifica como La sostenido -A#-, que son la misma nota). Esto resulta coherente con la estructura armónica de la pieza y era de esperarse.

Para un análisis futuro podrían explorarse los acordes de toda la pieza. Además, podría realizarse un análisis de varias piezas de un autor en particular (o de un periodo musical).

Referencias

Chávez Cano, A. S. (2024). *Tiempos de mezcla en cadenas de Markov: aplicación al problema del barajeo de cartas* (Tesis de licenciatura, Instituto Tecnológico Autónomo de México). Biblioteca Raúl Baillères Jr., ITAM.

Smith, T. (2025). *Random Walk Theory*. Investopedia. Recuperado de <https://www.investopedia.com/terms/r/randomwalktheory.asp>

Vera, B., & Toloza, S. (2022). *Generación de música mediante cadenas de Markov*.

Malliaros, F. D. (2013). *Clustering and community detection in directed networks: A survey*. *Physics Reports*, 533(4), 95–142. Elsevier. Recuperado de <https://www.sciencedirect.com/science/article/abs/pii/S0370157313002822>

Random walk: An overview. (s. f.). En *Topics in Physics and Astronomy*. Recuperado de <https://www.sciencedirect.com/topics/physics-and-astronomy/random-walk>

1. Anexo

A. Código Python para la Simulación y Visualización de la Cadena

A continuación se presenta el script completo en Python utilizado para simular la trayectoria de una cadena de nacimiento y muerte y para generar el grafo de su espacio de estados. El código requiere las librerías `numpy`, `matplotlib` y `networkx`.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import networkx as nx
4
5 def simulate_birth_death_chain(N, p_probs, q_probs, start_state, num_steps
6 ):
7     """
8     Simula una trayectoria de una cadena de nacimiento y muerte.
9     """
10    if not (len(p_probs) == N + 1 and len(q_probs) == N + 1):
11        raise ValueError("Las listas de probabilidades deben tener N+1
12 elementos.")
13
14    r_probs = [1 - p - q for p, q in zip(p_probs, q_probs)]
15
16    current_state = start_state
17    path = [current_state]
18
19    for _ in range(num_steps):
20        i = current_state
21        p_i, q_i, r_i = p_probs[i], q_probs[i], r_probs[i]
22        rand_num = np.random.rand()
23
24        if rand_num < p_i:
25            current_state = min(i + 1, N) # Nacimiento
26        elif rand_num < p_i + q_i:
27            current_state = max(i - 1, 0) # Muerte
28        else:
29            current_state = i # Estasis
30
31        path.append(current_state)
32
33    return path
```

```

33 def plot_state_space_graph(N, p_probs, q_probs, filename="
state_space_graph_improved.png"):
34     """
35     Genera y guarda una visualizacion MEJORADA y mas legible del grafo del
espacio de estados.
36     """
37     G = nx.DiGraph()
38     r_probs = [1 - p - q for p, q in zip(p_probs, q_probs)]
39
40     # Anadir nodos y aristas con sus probabilidades
41     G.add_nodes_from(range(N + 1))
42     for i in range(N + 1):
43         if p_probs[i] > 0: G.add_edge(i, i + 1, label=f'p_{i}={p_probs[i]
:.2f}', type='p')
44         if q_probs[i] > 0: G.add_edge(i, i - 1, label=f'q_{i}={q_probs[i]
:.2f}', type='q')
45         if r_probs[i] > 0: G.add_edge(i, i, label=f'r_{i}={r_probs[i]:.2f}
', type='r')
46
47     # Posicionar los nodos en una linea horizontal
48     pos = {i: (i, 0) for i in range(N + 1)}
49
50     plt.figure(figsize=(20, 7))
51
52     # Dibujar nodos y sus etiquetas numericas
53     nx.draw_networkx_nodes(G, pos, node_size=800, node_color='skyblue')
54     nx.draw_networkx_labels(G, pos, font_size=14, font_color='black')
55
56     # Separar aristas de nacimiento y muerte para dibujarlas con diferente
curvatura
57     forward_edges = [(u, v) for u, v, d in G.edges(data=True) if d['type']
== 'p']
58     backward_edges = [(u, v) for u, v, d in G.edges(data=True) if d['type']
== 'q']
59     self_loops = [(u, v) for u, v, d in G.edges(data=True) if d['type'] ==
'r']
60
61     # Dibujar aristas de nacimiento (p) con un arco hacia arriba
62     nx.draw_networkx_edges(G, pos, edgelist=forward_edges, width=1.5,
alpha=0.8, edge_color='gray',
63                             connectionstyle='arc3,rad=0.2', arrowsize=20)
64
65     # Dibujar aristas de muerte (q) con un arco hacia abajo
66     nx.draw_networkx_edges(G, pos, edgelist=backward_edges, width=1.5,
alpha=0.8, edge_color='gray',

```

```

67         connectionstyle='arc3,rad=-0.2', arrowsize=20)
68
69     # Dibujar self-loops de forma mas pronunciada
70     nx.draw_networkx_edges(G, pos, edgelist=self_loops, width=1.0, alpha
71                             =0.7, edge_color='salmon',
72                             connectionstyle='arc3,rad=0.8', arrowsize=20,
73                             arrowstyle='->')
74
75     # Posicionar etiquetas de forma inteligente
76     p_labels = { (u,v): d['label'] for u,v,d in G.edges(data=True) if d['
77 type'] == 'p' }
78     q_labels = { (u,v): d['label'] for u,v,d in G.edges(data=True) if d['
79 type'] == 'q' }
80     r_labels = { (u,v): d['label'] for u,v,d in G.edges(data=True) if d['
81 type'] == 'r' }
82
83     # Dibujar etiquetas 'p' por encima de la linea de nodos
84     nx.draw_networkx_edge_labels(G, pos, edge_labels=p_labels, font_size
85 =10,
86                                 bbox=dict(facecolor='white', alpha=0,
87 edgecolor='none'), verticalalignment='bottom')
88
89     # Dibujar etiquetas 'q' por debajo de la linea de nodos
90     nx.draw_networkx_edge_labels(G, pos, edge_labels=q_labels, font_size
91 =10,
92                                 bbox=dict(facecolor='white', alpha=0,
93 edgecolor='none'), verticalalignment='top')
94
95     # Dibujar etiquetas 'r' (self-loops) mas arriba
96     for node, loop_edge in zip(G.nodes, self_loops):
97         nx.draw_networkx_edge_labels(G, pos, edge_labels={(loop_edge):
98 r_labels[loop_edge]},
99                                     font_size=10, label_pos=0.5,
100                                     bbox=dict(facecolor='white', alpha=0,
101 edgecolor='none'),
102                                     verticalalignment='bottom',
103                                     horizontalalignment='center',
104                                     font_color='darkred')
105
106     plt.title("Grafo del Espacio de Estados", fontsize=18)
107     plt.xlim(-0.5, N + 0.5)
108     plt.ylim(-0.5, 1)
109     plt.box(False)
110     plt.tight_layout()
111     plt.savefig("state_space_graph_improved.png")

```

```

100     plt.show()
101     print(f"Grafo mejorado guardado como 'state_space_graph_improved.png'"
102         )
103
104 # --- Ejemplo de uso ---
105 # Definir parametros para una caminata aleatoria con barreras absorbentes
106 N_sim = 10
107 p = 0.6
108 q = 0.4
109
110 # Barreras absorbentes en 0 y N
111 p_sim = [0] + [p] * (N_sim - 1) + [0]
112 q_sim = [0] + [q] * (N_sim - 1) + [0]
113
114 # 1. Simular la trayectoria
115 trayectoria = simulate_birth_death_chain(N_sim, p_sim, q_sim, start_state
116     =5, num_steps=50)
117 print(f"Trayectoria simulada:\n{trayectoria}\n")
118
119 # 2. Generar y mostrar el nuevo grafo del espacio de estados
120 plot_state_space_graph(N_sim, p_sim, q_sim)

```

Listing 1: Script de Python para simulación y graficación.